Számítógépes Hálózatok

6. gyakorlat

Ellenőrző kódok:

CRC, MD5

CRC

- Definiáljuk a G(x) generátor polinomot (G foka r), amelyet a küldő és a vevő egyaránt ismer.
- **Algoritmus:**

1. Legyen G(x) foka r. Fűzzünk r darab 0 bitet a keret alacsony helyi értékű végéhez, így az m+r bitet fog tartalmazni és az $x^rM(x)$ polinomot fogja reprezentálni.

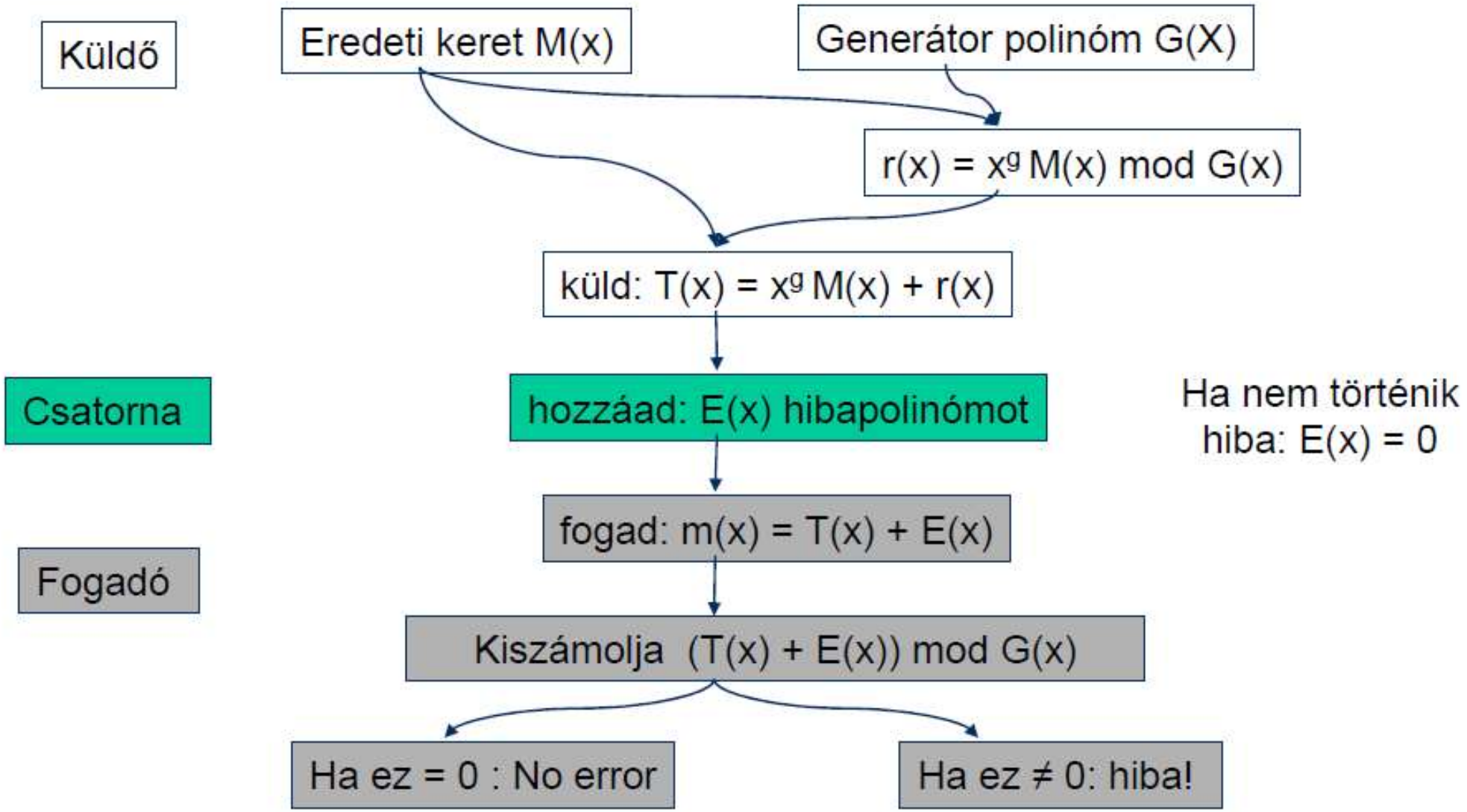
2. Osszuk el az $x^rM(x)$-hez tartozó bitsorozatot a G(x)-hez tartozó bitsorozattal modulo 2

3. Vonjuk ki a maradékot (mely mindig r vagy kevesebb bitet tartalmaz) az $x^rM(x)$-hez tartozó bitsorozatból. Az eredmény az ellenőrző összeggel ellátott, továbbítandó keret. Jelölje a továbbítandó keretnek megfelelő a polinomot T(x).

4. A vevő a T(x) + E(x) polinomnak megfelelő sorozatot kapja, ahol E(x) a hiba polinom. Ezt elosztja G(x) generátor polinommal.

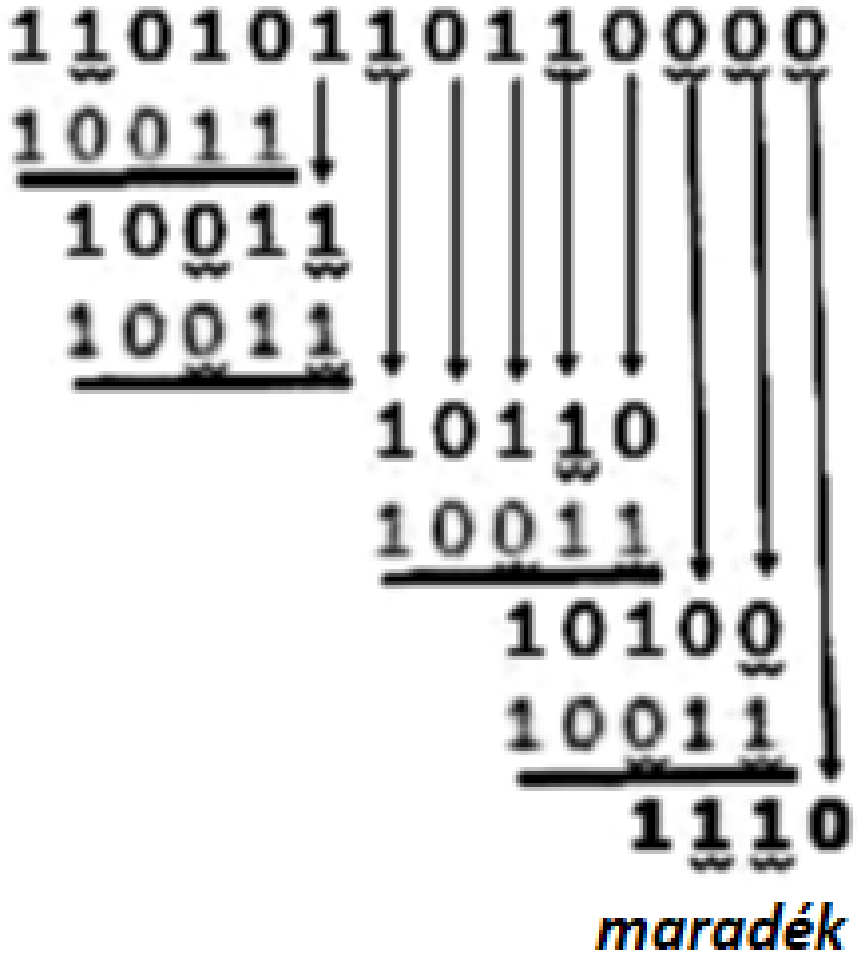
- Ha az osztási maradék, amit R(x) jelöl, nem nulla, akkor hiba történt

CRC


Küldő
Eredeti keret M(x)
Generátor polinóm G(X)
r(x) = x^g M(x) mod G(x)
küld: T(x) = x^g M(x) + r(x)
Csatorna
hozzáad: E(x) hibapolinómot
Ha nem történik hiba: E(x) = 0
fogad: m(x) = T(x) + E(x)
Fogadó
Kiszámolja (T(x) + E(x)) mod G(x)
Ha ez = 0 : No error
Ha ez ≠ 0: hiba!

CRC példa

- Keret: 1101011011
- Generátor: 10011 -> (x^4+x+1)
  4 fokú polinom
- Kiegészített keret:
  11010110110000
- Osztás binárisan: XOR művelet
- Maradék: 1110
- A továbbítandó üzenet:
  11010110111110



CRC példa

- Az osztásban 11010110110000 az $x^{13}+x^{12}+x^{10}+x^8+x^7+x^5+x^4$ polinomot reprezentálja. 10011 pedig az x^4+x+1 polinomot.
- Ebből (11010110110000) * (1110) = 11010110111110 ami az elküldendő keretünk lesz.
- (11010110111110) mod 10011 = 0
- Amennyiben hozzáadtunk volna egy E(x) hibapolinomot (pl. $E(x) = x^2+x$ =110), akkor a maradék nem nulla (a példában 11) lenne, így tudnánk, hogy meghibásodott a keret.

CRC, MD5 pythonban

- CRC

```
import binascii, zlib

test_string = "Fekete retek rettenetes".encode('utf-8')

print(hex(binascii.crc32(bytearray(test_string))))
print(hex(zlib.crc32(test_string)))
```

- MD5

```
import hashlib

test_string = "Fekete retek rettenetes".encode('utf-8')

m = hashlib.md5()
m.update(test_string)
print(m.hexdigest())
```

PYTHON SOCKET - PROXY

Bemutató feladat I.

Készítsünk egy egyszerű TCP alapú proxyt (átjátszó). A proxy a kliensek felé szerverként látszik, azaz a kliensek csatlakozhatnak hozzá. A proxy a csatlakozás után kapcsolatot nyit egy szerver felé (parancssori argumentum), majd minden a klienstől jövő kérést továbbítja a szerver felé és a szervertől jövő válaszokat pedig a kliens felé.

Futtatása pl: netProxy.py ggombos.web.elte.hu 80

Ez alkalommal ne készítsünk saját klienset, hanem használjuk egy webböngészőt.

A proxy használja a localhost:10000 -es socket address-t.

Web böngészőbe írjuk be: localhost:10000

Bemutató feladat II. – Tiltsunk le valamilyen tartalmat

A SzamHalo-t tartalmazó URL-ek ne legyenek elérhetők a proxyn keresztül.

A válasz legyen valamilyen egyszerű HTML üzenet, ami jelzi a blokkolást.

megj: header: „HTTP/1.1 404 Not Found\n\n”

Proxy

Felhasználásai:

- adat továbbítás
  (pl. el akarjuk rejteni a rendes szervert.)
- folyam vezérlés és kiegyenlítés
  (pl. hogy elosztott szerverek egyenletesen kapják a kéréseket.)
- adat cache-elés
  (pl. lehal a szerver, de a proxy ideiglenesen/statikusan kiszolgálja a (le)kéréseket.)
- stb...

Veszélyei:

- adatlopás
  (pl. a proxy egy nagy cég honlapját játssza át hasonló domain névvel és ugyan úgy használható mint az eredeti, viszont lementi a begépelt jelszavakat)
- stb...

Számológép Proxy

- Készítsünk a számológéphez egy proxy-t, ami értelmezi a klienstől kapott TCP kéréseket és "szabotálja" azokat.
- Szabotálás: az első operandust megszorozza kettővel, a másodikhoz pedig hozzáad egyet.
- A módosított kérést elküldi a TCP servernek, majd az eredményt visszaküldi a kliensnek.

Ismétlés

FÁJLÁTVITEL (SZÖVEG ÉS KÉP)

Fájl átvitel

- fájl bináris megnyitása

```
with open („input.txt”, „rb”) as f:
        …
```

- read(x) – x bytes

```
        …
        f.read(128)    #128 byte-ot fog beolvasni
```

„When size is omitted or negative, the entire contents of the file will be read and returned; it’s your problem if the file is twice as large as your machine’s memory. „ - python.org

Feladat - Bináris fájl átvitel

- Juttassunk át egy szöveget tartalamzó fájlt (input.txt) a kilensről a szerverre bináris módon. (Ez most lokálisan egy fájl másolásnak fog megfelelni.) Az új fájl neve legyen output.txt.

- Egészítsük ki a kódot úgy, hogy egy bármilyen fájlformátumot (pl. png kép) is át tudjunk küldeni. A kliens paramétere az eredeti fájl, a szerver paramétere az új létrejövő fájl neve legyen.

BEADANDÓ IV. (1 PONT)

Beadandó
netcopy alkalmazás 1 pont

Készíts egy netcopy kliens/szerver alkalmazást, mely egy fájl átvitelét és az átvitt adat ellenőrzését teszi lehetővé MD5 ellenőrzőösszeg segítségével! A feladat során három komponenst/programot kell elkészíteni:

1. **Netcopy kliens**: egy parancssori argumentumban megadott fájlt átküld a szervernek. Az átvitel során/végén kiszámol egy md5 checksumot a fájlra, majd ezt feltölti fájl azonosítóval együtt a Checksum szerverre. A lejárati idő 60 mp. A fájl azonosító egy egész szám, amit szintén parancssori argumentumban kell megadni.
2. **Netcopy szerver**: Vár, hogy egy kliens csatlakozzon. Csatlakozás után fogadja az átvitt bájtokat és azokat elhelyezi a parancssori argumentumban megadott fájlba. A végén lekéri a Checksum szervertől a fájl azonosítóhoz tartozó md5 checksumot és ellenőrzi az átvitt fájl helyességét, melynek eredményét stdoutputra is kiírja. A fájl azonosító itt is parancssori argumentum kell legyen.
3. **Checksum szerver**: (fájl azonosító, checksum hossz, checksum, lejárat (mp-ben)) négyesek tárolását és lekérdezését teszi lehetővé. A protokoll részletei a következő oldalon.

Beadás: TMS-ben

Netcopy kliens – **TCP alapú**

- Működés:
  - Csatlakozik a szerverhez, aminek a címét portját parancssori argumentumban kapja meg.
  - Fájl bájtjainak sorfolytonos átvitele a szervernek.
  - A Checksum szerverrel az ott leírt módon kommunikál.
  - A fájl átvitele és a checksum elhelyezése után bontja a kapcsolatot és terminál.
- Futtatás:
  - ./netcopy_cli.py <srv_ip> <srv_port> <chsum_srv_ip> <chsum_srv_port> <fájl azon> <fájlnév elérési úttal>
    - <fájl azon>: egész szám
    - <srv_ip> <srv_port>: a netcopy szerver elérhetősége
    - <chsum_srv_ip> <chsum_srv_port>: a Checksum szerver elérhetősége

Netcopy szerver – **TCP alapú**

- Működés:
  - Bindolja a socketet a parancssori argumentumban megadott címre.
  - Vár egy kliensre.
  - Ha acceptálta, akkor fogadja a fájl bájtjait sorfolytonosan és kiírja a paracssori argumentumban megadott fájlba.
  - Fájlvége jel olvasása esetén lezárja a kapcsolatot és utána ellenőrzi a fájlt a Checksum szerverrel.
  - A Checksum szerverrel az ott leírt módon kommunikál.
  - Hiba esetén a stdout-ra ki kell írni: CSUM CORRUPTED
  - Helyes átvitel esetén az stdout-ra ki kell írni: CSUM OK
  - Fájl fogadása és ellenőrzése után terminál a program.
- Futtatás:
  - ./netcopy_srv.py <srv_ip> <srv_port> <chsum_srv_ip> <chsum_srv_port> <fájl azon> <fájlnév elérési úttal>
    - <fájl azon>: egész szám ua. mint a kliensnél – ez alapján kéri le a szervertől a checksumot
    - <srv_ip> <srv_port>: a netcopy szerver elérhetősége – bindolásnál kell
    - <chsum_srv_ip> <chsum_srv_port>: a Checksum szerver elérhetősége
    - <fájlnév> : ide írja a kapott bájtokat

Checksum szerver - TCP

- Beszúr üzenet
  - Formátum: szöveges
  - Felépítése: BE|<fájl azon.>|<érvényesség másodpercben>|<checksum hossza bájtszámban>|<checksum bájtjai>
  - A „|” delimiter karakter
  - Példa: BE|1237671|60|12|abcdefabcdef
    - Ez esetben: a fájlazon: 1237671, 60mp az érvényességi idő, 12 bájt a checksum, abcdefabcdef maga a checksum
  - Válasz üzenet: OK
- Kivesz üzenet
  - Formátum: szöveges
  - Felépítése: KI|<fájl azon.>
  - A „|” delimiter karakter
  - Példa: KI|1237671
    - Azaz kérjük az 1237671 fájl azonosítóhoz tartozó checksum-ot
  - Válasz üzenet: <checksum hossza bájtszámban>|<checksum bájtjai>
    Péda: 12|abcdefabcdef
  - Ha nincs checksum, akkor ezt küldi: 0|
- Futtatás
  - ./checksum_srv.py <ip> <port>
    - <ip> - pl. localhost a szerver címe bindolásnál
    - <port> - ezen a porton lesz elérhető
  - A szerver végtelen ciklusban fut és egyszerre több klienst is ki tud szolgálni. A kommunikáció TCP, csak a fenti üzeneteket kezeli.
  - Lejárat utáni checksumok törlődnek.

Órai feladat

Órai feladat - Fájl átvitel ellenőrzése

- Küldjünk át binárisan egy fájlt a parancssori argumentumról a kliensről a szerverre.
- A kliens számoljon ki a fájlból egy md5-ös kódot, majd azt is küldje át a szervernek.
- A szerver fogadja a kódot is és a kapott fájlra ő is számoljon ki egy md5-ös kódot.
- Majd a szerver vesse össze a két md5-ös kódot, ha megegyezik, akkor írjaki stdout-ra, hogy "OK", ha nem akkor pedig "HIBA".

Szorgalmi feladat otthonra – fájlátvitel UDP felett

Fájlátvitel megvalósítása úgy, hogy a fájl letöltése UDP felett legyen megoldva. Készüljünk fel arra, hogy az átvitel során csomagvesztés, vagy sorrend csere is történhet! Az UDP szerver portját szabadon definiálhatjuk!

A hibakezeléshez egy javaslat:

Max. 1000 bájtonként UDP csomagokban elkezdjük átküldeni a fájl tartalmát. Minden csomag egy pár bájtos fejléccel indul, amiben jelezzük, hogy az utolsó darab-e, amit átküldtünk, továbbá egy másik mező jelzi a byteoffset-et a fájl elejétől. Működés:

- Ha a kliens kapott egy adatcsomagot, akkor egy nyugtacsomagot küld vissza.
- A nyugtacsomag fogadása után a szerver, küldi a következő adatcsomagot.
- Ha nem jön nyugta, akkor T idő után újraküldi a korábbi adatcsomagot. (pl. T=200ms)
- Ha nyugta veszik el, akkor a vevő az offset alapján el tudja dönteni, hogy egy új adatcsomag, vagy egy korábbi duplikátuma érkezett-e.
- Ha az utolsó csomag is megérkezett, akkor a kliens nyugtázza azt is és lezárja a fájlba írást. A szerver az utolsó nyugta után befejezi az átvitelt.

VÉGE

KÖSZÖNÖM A FIGYELMET!